

GUÍA PRÁCTICA - REACT PARTE II

Desarrollo Fullstack II - DSY1104

Proyecto	Tienda Fullstack
Objetivo	Continuar el proyecto React y aplicar diseño responsivo, estado, persistencia, rutas, parámetros y formularios.
Entorno	React + Vite + Bootstrap + React Router
Resultado	SPA con Inicio, Productos, Detalle de Producto y Login, incluyendo stock persistente y validación.

Importante: esta guía continúa el proyecto realizado en la clase anterior. No se crea un proyecto nuevo. Se asume que ya existen **Producto.jsx**, **Contador.jsx** y un proyecto Vite funcionando.

Guía paso a paso

1. Levantar el proyecto

Abra en Visual Studio Code el proyecto utilizado en la clase anterior. Desde la terminal integrada ejecute:

```
npm run dev
```

Abra la dirección indicada por Vite, normalmente:

```
http://localhost:5173/
```

Verifique que el proyecto anterior funcione antes de continuar.

2. Repaso: Atomic Design

Atomic Design propone construir interfaces a partir de elementos pequeños y reutilizables que luego se combinan en estructuras mayores.

Nivel	Ejemplo en nuestra tienda
Átomo	Botón Comprar / botón + / botón -
Molécula	Nombre + precio + botón
Organismo	Tarjeta completa de Producto
Plantilla	Navbar + zona de productos + estructura general
Página	Tienda con contenido real

3. Instalar Bootstrap y preparar diseño responsivo

Instale Bootstrap desde la terminal:

```
npm install bootstrap
```

En `src/main.jsx` importe Bootstrap:

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import 'bootstrap/dist/css/bootstrap.min.css'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

3.1 Actualizar Producto.jsx

Mantendremos el estado de stock y agregaremos clases Bootstrap.

```
import { useState } from 'react'

function Producto({ nombre, precio }) {

  const [stock, setStock] = useState(5)

  const disminuirStock = () => {
    if (stock > 0) {
      setStock(stock - 1)
    }
  }

  return (
    <div className="card h-100">
      <div className="card-body">
        <h5 className="card-title">{nombre}</h5>
        <p className="card-text">Precio: ${precio}</p>
        <p>Stock disponible: {stock}</p>

        <button
          className="btn btn-danger me-2"
          onClick={disminuirStock}
        >

```

```

        >
        -
      </button>

      <button
        className="btn btn-success"
        onClick={() => setStock(stock + 1)}
      >
        +
      </button>
    </div>
  </div>
)
}

export default Producto

```

3.2 Responsive en App.jsx

```

import Producto from './components/Producto'

function App() {
  return (
    <div className="container mt-4">
      <h1 className="text-center mb-4">Tienda Fullstack</h1>

      <div className="row g-4">
        <div className="col-12 col-md-6 col-lg-4">
          <Producto nombre="Notebook Lenovo" precio="599.990" />
        </div>

        <div className="col-12 col-md-6 col-lg-4">
          <Producto nombre="Mouse Logitech" precio="29.990" />
        </div>

        <div className="col-12 col-md-6 col-lg-4">
          <Producto nombre="Teclado Redragon" precio="49.990" />
        </div>
      </div>
    </div>
  )
}

export default App

```

Repaso Bootstrap: col-12 ocupa 12/12 columnas (100%); col-md-6 ocupa 6/12 (50%); col-lg-4 ocupa 4/12 (33,3%). Por lo tanto: celular = 1 producto por fila, tablet = 2 y pantalla grande = 3.

4. Persistencia con localStorage

Si cambiamos el stock y presionamos F5, el estado vuelve al valor inicial. Para conservarlo utilizaremos localStorage.

4.1 Conceptos básicos

```

localStorage.setItem('nombre', 'Juan')
localStorage.getItem('nombre')
localStorage.removeItem('nombre')

```

localStorage conserva datos en el navegador. Los valores almacenados se recuperan como texto.

4.2 Guardar y recuperar el stock

Actualice **Producto.jsx**. Agregamos **useEffect** para guardar cada cambio y una inicialización que recupera el stock existente.

```

import { useState, useEffect } from 'react'

function Producto({ nombre, precio }) {

  const [stock, setStock] = useState(() => {
    const stockGuardado = localStorage.getItem(nombre)

    return stockGuardado !== null
  })

```

```

        ? Number(stockGuardado)
        : 5
    })

    useEffect(() => {
        localStorage.setItem(nombre, stock)
    }, [stock, nombre])

    const disminuirStock = () => {
        if (stock > 0) {
            setStock(stock - 1)
        }
    }

    return (
        <div className="card h-100">
            <div className="card-body">
                <h5 className="card-title">{nombre}</h5>
                <p className="card-text">Precio: ${precio}</p>
                <p>Stock disponible: {stock}</p>

                <button
                    className="btn btn-danger me-2"
                    onClick={disminuirStock}
                >
                    -
                </button>

                <button
                    className="btn btn-success"
                    onClick={() => setStock(stock + 1)}
                >
                    +
                </button>
            </div>
        </div>
    )
}

export default Producto

```

Prueba: cambie los stocks de los productos, presione F5 y compruebe que los valores permanecen. Number() convierte el texto recuperado desde localStorage a número.

sessionStorage: su uso es similar, pero la información dura durante la sesión de la pestaña. Ejemplo: sessionStorage.setItem('nombre','Juan').

5. React Router: crear una SPA con varias rutas

Instale React Router:

```
npm install react-router-dom
```

Cree una carpeta **src/pages** con estos archivos:

```

src/
├── components/
│   ├── Producto.jsx
├── pages/
│   ├── Inicio.jsx
│   ├── Productos.jsx
│   ├── Login.jsx
├── App.jsx
└── main.jsx

```

5.1 Inicio.jsx

```

function Inicio() {
    return (
        <div className="container mt-4">
            <h1>Bienvenido a Tienda Fullstack</h1>
            <p>Nuestra primera aplicación utilizando React.</p>
        </div>
    )
}

```

```
export default Inicio
```

5.2 Productos.jsx

```
import Producto from '../components/Producto'

function Productos() {
  return (
    <div className="container mt-4">
      <h1 className="text-center mb-4">Nuestros Productos</h1>

      <div className="row g-4">
        <div className="col-12 col-md-6 col-lg-4">
          <Producto nombre="Notebook Lenovo" precio="599.990" />
        </div>

        <div className="col-12 col-md-6 col-lg-4">
          <Producto nombre="Mouse Logitech" precio="29.990" />
        </div>

        <div className="col-12 col-md-6 col-lg-4">
          <Producto nombre="Teclado Redragon" precio="49.990" />
        </div>
      </div>
    </div>
  )
}

export default Productos
```

5.3 Login.jsx inicial

```
function Login() {
  return (
    <div className="container mt-4">
      <h1>Iniciar Sesión</h1>
      <p>Aquí construiremos nuestro formulario de Login.</p>
    </div>
  )
}

export default Login
```

5.4 Configurar rutas en App.jsx

```
import { BrowserRouter, Routes, Route } from 'react-router-dom'

import Inicio from './pages/Inicio'
import Productos from './pages/Productos'
import Login from './pages/Login'

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Inicio />} />
        <Route path="/productos" element={<Productos />} />
        <Route path="/login" element={<Login />} />
      </Routes>
    </BrowserRouter>
  )
}

export default App
```

Pruebe manualmente `/`, `/productos` y `/login`. **BrowserRouter** habilita la navegación; **Routes** agrupa las rutas; cada **Route** relaciona una URL con un componente.

6. Crear Navbar y navegar con Link

Cree `src/components/Navbar.jsx`:

```
import { Link } from 'react-router-dom'
```

```
function Navbar() {
  return (
    <nav className="navbar navbar-expand-lg bg-dark">
      <div className="container">

        <Link className="navbar-brand text-white" to="/">
          Tienda Fullstack
        </Link>

        <div>
          <Link className="btn btn-outline-light me-2" to="/">
            Inicio
          </Link>

          <Link className="btn btn-outline-light me-2" to="/productos">
            Productos
          </Link>

          <Link className="btn btn-outline-light" to="/login">
            Login
          </Link>
        </div>
      </div>
    </nav>
  )
}

export default Navbar
```

Ahora modifique **App.jsx** para incorporar el Navbar:

```
import { BrowserRouter, Routes, Route } from 'react-router-dom'

import Navbar from './components/Navbar'
import Inicio from './pages/Inicio'
import Productos from './pages/Productos'
import Login from './pages/Login'

function App() {
  return (
    <BrowserRouter>

      <Navbar />

      <Routes>
        <Route path="/" element={<Inicio />} />
        <Route path="/productos" element={<Productos />} />
        <Route path="/login" element={<Login />} />
      </Routes>

    </BrowserRouter>
  )
}

export default App
```

Link permite navegar dentro de la SPA sin utilizar la navegación tradicional de un enlace HTML. El Navbar se coloca fuera de Routes para que aparezca en todas las páginas.

7. Rutas con parámetros: useParams

Queremos una sola página de detalle capaz de responder a /producto/1, /producto/2, etc.

7.1 Crear DetalleProducto.jsx

```
import { useParams } from 'react-router-dom'

function DetalleProducto() {

  const { id } = useParams()

  const productos = {
    1: 'Notebook Lenovo',
    2: 'Mouse Logitech',
    3: 'Teclado Redragon'
```

```

    }

    const nombreProducto = productos[id]

    return (
      <div className="container mt-4">
        <h1>Detalle del Producto</h1>
        <p>Producto seleccionado: {id}</p>
        <h3>{nombreProducto}</h3>
      </div>
    )
  }
}

export default DetalleProducto

```

7.2 Agregar la ruta dinámica

En **App.jsx** importe el componente:

```
import DetalleProducto from './pages/DetalleProducto'
```

Y agregue dentro de **<Routes>**:

```

<Route
  path="/producto/:id"
  element={<DetalleProducto />}
/>

```

:id indica que esa parte de la URL es variable. `useParams()` permite recuperar ese valor.

7.3 Enviar el id desde Productos.jsx

Agregue un id a cada producto:

```

<Producto id="1" nombre="Notebook Lenovo" precio="599.990" />
<Producto id="2" nombre="Mouse Logitech" precio="29.990" />
<Producto id="3" nombre="Teclado Redragon" precio="49.990" />

```

7.4 Agregar 'Ver detalle' en Producto.jsx

Cambie la firma del componente:

```
function Producto({ id, nombre, precio }) {
```

Importe Link:

```
import { Link } from 'react-router-dom'
```

Y agregue junto a los botones:

```

<Link
  to={`/${producto}/${id}`}
  className="btn btn-primary ms-2"
>
  Ver detalle
</Link>

```

Pruebe los tres productos. Cada botón debe llevar a su URL correspondiente y **DetalleProducto.jsx** debe mostrar el id y el nombre.

8. Parámetros de búsqueda: `useLocation`

Ahora leeremos una query string como `/productos?categoria=computacion`.

En **Productos.jsx** importe:

```
import { useLocation } from 'react-router-dom'
```

Dentro del componente agregue:

```

const location = useLocation()
const parametros = new URLSearchParams(location.search)
const categoria = parametros.get('categoria')

```

Y antes de las tarjetas muestre:

```
<p>
  Categoría seleccionada: {categoria}
</p>
```

Pruebe:

```
http://localhost:5173/productos?categoria=computacion
http://localhost:5173/productos?categoria=accesorios
```

Diferencia: useParams recupera valores que forman parte de la ruta; useLocation permite acceder a la ubicación actual y a la cadena de búsqueda de la URL.

9. Formulario controlado y validación

Finalmente transformaremos **Login.jsx** en un formulario controlado. React mantendrá los valores de los campos en estados.

```
import { useState } from 'react'

function Login() {

  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [error, setError] = useState('')

  const handleSubmit = (event) => {

    event.preventDefault()

    if (!email.includes('@')) {
      setError('El correo electrónico no es válido')
      return
    }

    if (password.length < 4) {
      setError('La contraseña debe tener al menos 4 caracteres')
      return
    }

    setError('')

    console.log('Email:', email)
    console.log('Password:', password)
  }

  return (
    <div className="container mt-4">

      <h1>Iniciar Sesión</h1>

      <form onSubmit={handleSubmit}>

        <div className="mb-3">
          <label className="form-label">
            Correo electrónico
          </label>

          <input
            type="text"
            className="form-control"
            value={email}
            onChange={(event) => setEmail(event.target.value)}
          />
        </div>

        <div className="mb-3">
          <label className="form-label">
            Contraseña
          </label>

          <input
            type="password"
            className="form-control"
            value={password}
            onChange={(event) => setPassword(event.target.value)}
          />
        </div>

      </form>

    </div>
  )
}
```

```

        </div>

        {error && (
          <div className="alert alert-danger">
            {error}
          </div>
        )}

        <button
          type="submit"
          className="btn btn-primary"
        >
          Ingresar
        </button>

      </form>

    </div>
  )
}

export default Login

```

9.1 Conceptos clave del formulario

Elemento	Función
useState	Mantiene los datos del formulario en React.
value	Vincula el valor visible del input con el estado.
onChange	Se ejecuta cada vez que cambia el campo.
event.target.value	Obtiene lo escrito por el usuario.
onSubmit	Se ejecuta al enviar el formulario.
preventDefault()	Evita el comportamiento tradicional de envío/recarga.
error && ...	Renderizado condicional: muestra el mensaje solo si existe un error.

9.2 Pruebas

- **Email:** hernan / **Password:** 12345 -> debe indicar correo inválido.
- **Email:** hernan@gmail.com / **Password:** 12 -> debe indicar contraseña demasiado corta.
- **Email:** hernan@gmail.com / **Password:** 12345 -> no debe mostrar error y debe escribir los valores en la consola.

10. Estructura final esperada

```

src/
├── components/
│   ├── Contador.jsx
│   ├── Navbar.jsx
│   ├── Producto.jsx
│   └── Saludo.jsx
├── pages/
│   ├── DetalleProducto.jsx
│   ├── Inicio.jsx
│   ├── Login.jsx
│   └── Productos.jsx
├── App.jsx
├── main.jsx
└── index.css

```

11. Lista de verificación final

- El proyecto inicia con npm run dev.
- Bootstrap está importado en main.jsx.

- ■ Los productos se adaptan a celular, tablet y pantalla grande.
- ■ El stock no puede bajar de 0.
- ■ El stock se conserva al actualizar gracias a localStorage.
- ■ El Navbar permite navegar a Inicio, Productos y Login.
- ■ React Router maneja las rutas sin crear una aplicación diferente.
- ■ Ver detalle navega a /producto/:id.
- ■ DetalleProducto obtiene el id mediante useParams.
- ■ Productos puede leer ?categoria=... mediante useLocation.
- ■ Login utiliza un formulario controlado con useState.
- ■ El formulario evita el envío tradicional con preventDefault().
- ■ Se muestran mensajes de validación cuando los datos son incorrectos.

12. Próxima actividad: AWS

En la siguiente sesión se aplicarán estos conceptos en el entorno AWS/EC2 siguiendo la guía de la asignatura. La actividad de AWS continúa con Bootstrap, formularios y posteriormente React Hook Form. React Hook Form se deja intencionalmente para esa sesión.

Consejo: antes de comenzar AWS, confirme que este proyecto local funciona completamente. De esta forma podrá distinguir un problema de React de un problema de configuración del entorno.